

1 System Implementation

The Wassit AI system follows a modular, full-stack architecture enforcing separation of concerns across presentation, business logic, and data layers. All components are containerised via Docker and orchestrated with Docker Compose.

1.1 Frontend Implementation

Built with React 18 and Vite, the frontend is split into two functional domains: the chat interface and the profile dashboard.

Technology stack: TailwindCSS for styling, Headless UI + Heroicons for accessible components, Axios for HTTP communication, and React Context with custom hooks for state management.

Component architecture:

- **Chat layer:** real-time message display, streaming response rendering, animated typing indicator.
- **Profile layer:** profile creation form, skills entry interface, experience timeline.
- **Common layer:** navbar, loading spinners, error boundary.

A dedicated services layer centralises API calls and session token management. Feature-specific hooks encapsulate side effects.

Performance optimisations: route-level lazy loading, debounced chat input, virtual scrolling for long histories, memoisation of expensive components.

1.2 Backend Implementation

Flask 2.3 REST API on port 5000, following a layered architecture.

Route layer: chat routes (message submission, history retrieval), profile routes (CRUD on user data), health routes (monitoring endpoints).

Service layer: chat service (LLM pipeline orchestration), recommendation service (job suggestions from profile + conversation), profile service (business rule enforcement).

Machine learning layer:

- Groq client wrapper for Mixtral-8x7b (general) and Llama 3 70b (complex tasks) with exponential backoff.
- Prompt engineering module with structured templates injecting profile, conversation, and job data.
- Intent classifier to route requests (job search, profile update, general chat) before LLM calls.

API endpoints:

Table 1: Profile, Chat, Job, and Health API Endpoints

Method	Endpoint	Description
POST	/api/profile	Create/update profile
GET	/api/profile/:id	Fetch profile
PUT	/api/profile/:id/skills	Update skills
POST	/api/chat	Send message to AI
GET	/api/chat/history/:userId	Retrieve history
DELETE	/api/chat/:sessionId	Clear session
GET	/api/jobs/recommend/:userId	Job recommendations
GET	/api/jobs/search	Search jobs
POST	/api/jobs/apply/:jobId	Apply to job
GET	/health	Health check
GET	/metrics	Performance metrics

Security: CORS restriction, Marshmallow validation, SQLAlchemy parameterised queries, CSP header, rate limiting (100 req/hour/client), environment variables for secrets.

1.3 Database Design

SQLite (development) with seamless migration to PostgreSQL. Six tables:

Table 2: Database Schema Overview

Table	Key Fields	Purpose
users	id, name, email, location, created_at	Root identity
skills	id, user_id, skill_name, proficiency_level	User skills
experience	id, user_id, job_title, company, start_date, end_date, description	Employment history
chat_sessions	id, user_id, session_token, created_at, last_active	Conversation groups
messages	id, session_id, role, content, timestamp	Message turns
jobs_cache	id, title, company, location, requirements, salary_range, posted_date	Cached job listings

Detailed schemas follow standard SQLAlchemy definitions with appropriate foreign keys and constraints.

1.4 Model Data & Input Structure

Wassit AI uses pre-trained LLMs (Mixtral-8x7b, Llama 3 70b) from Groq Cloud. Intelligence comes from three inference-time inputs:

1.4.1 User Profile Context

Includes name, location, skills (with proficiency), job titles, companies, dates, role descriptions, and conversation history (last N turns).

1.4.2 Jobs Dataset

Fields: title, company, location, requirements, salary_range, posted_date. Used for geographical and semantic matching.

1.4.3 Prompt Template

Dynamic assembly of system instructions, user profile, top-K job listings, conversation history, and current message — implementing Retrieval-Augmented Generation (RAG) to ground responses in real data and prevent hallucination.

1.5 Integration of Chatbot Components

1.5.1 Profile Creation Flow

Frontend collects profile data → backend validates → profile service writes user, skills, experience in one atomic transaction → confirmation propagates to frontend.

1.5.2 Chat Interaction Flow (8 steps)

1. User submits message; frontend shows optimistic bubble.
2. Chat route passes control to chat service.
3. Service retrieves full user profile from DB.
4. Intent classifier selects processing path.
5. Prompt engineering module assembles final LLM prompt.
6. Prompt submitted to Groq Cloud with streaming enabled.
7. Frontend renders tokens in real time.
8. Both user message and assistant reply persisted to database.

1.5.3 Authentication Integration

On profile creation, backend generates a session token stored locally and attached to subsequent requests. Token validated against `chat_sessions` table.

1.5.4 Deployment Integration

Three Docker containers: Flask backend (port 5000, Gunicorn), React frontend (port 3000), Nginx reverse proxy (port 80). Nginx routes `/api` to backend, all other traffic to frontend. Persistent Docker volume for SQLite database.

These mechanisms enable seamless movement from profile creation to AI-driven job matching, fulfilling the core design objective of an intelligent, context-aware employment assistant.